Introduction to Machine Learning: Supervised Learning

Abstract

These notes introduce supervised learning with neural networks, with physics-informed neural networks (PINNs) as the target application. We first state the problem: approximating an unknown function from a finite set of input–output examples by a parametric model. We then build the model used throughout, the multilayer perceptron (MLP), from Rosenblatt's perceptron, and justify it by the universal approximation theorem. Finally, we cast learning as the minimization of a loss function and review how to solve it in practice: data splitting, initialization, first-order methods (gradient descent, mini-batches, Momentum, RMSProp, Adam, learning-rate schedules) and second-order methods (Newton, L-BFGS and recent variants).

Contents

1 Introduction **1**
1.1 Supervised learning . . . 1
1.2 A first example: the linear model . . . 2
1.3 Notation . . . 2
1.4 Outline . . . 3

2 The model $\tilde{f}_{\boldsymbol{\theta}}$: neural networks **3**
2.1 From the perceptron to the neural unit . . . 3
2.2 One layer of neurons . . . 4
2.3 Activation functions . . . 4
2.4 The multilayer perceptron (MLP) . . . 5
2.5 Number of parameters . . . 6
2.6 Why MLPs? The universal approximation theorem . . . 7

3 Training: finding an optimal $\boldsymbol{\theta}$ **7**
3.1 Formulation . . . 7
3.2 Dataset: normalization and splitting . . . 8
3.3 Initialization of $\boldsymbol{\theta}_0$ . . . 9
3.4 Overview of gradient-based optimization methods . . . 10
3.5 First-order methods . . . 11
3.6 The learning rate and its schedule . . . 13
3.7 Second-order methods . . . 13
3.8 A worked example: GD, Adam and L-BFGS . . . 15
3.9 Mini-batches . . . 16
3.10 Training in practice . . . 18

1 Introduction

1.1 Supervised learning

We consider an unknown function

$$f : \mathbb{R}^n \to \mathbb{R}^m, \qquad \boldsymbol{x} \mapsto \boldsymbol{y} = f(\boldsymbol{x}),$$

which we can only observe through a finite dataset of N input–output pairs,

$$\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{y}_i\}_{i=1}^{N}, \qquad \boldsymbol{y}_i = f(\boldsymbol{x}_i).$$

In practice the outputs may be noisy measurements, $\boldsymbol{y}_i = f(\boldsymbol{x}_i) + \boldsymbol{\varepsilon}_i$.

Goal. Build a model $\tilde{f}$ such that $\tilde{f}(\boldsymbol{x}) \approx f(\boldsymbol{x})$, not only at the observed points $\boldsymbol{x}_i$ but also at new inputs $\boldsymbol{x}$.

Parametric model. Typically, the model is chosen within a family of functions indexed by a parameter vector: $\tilde{f} = \tilde{f}_{\boldsymbol{\theta}}$, with $\boldsymbol{\theta} \in \mathbb{R}^d$ the *model parameters*. Learning then amounts to choosing $\boldsymbol{\theta}$, a problem in $\mathbb{R}^d$ rather than in an infinite-dimensional space of functions.

1.2 A first example: the linear model

For $f : \mathbb{R} \to \mathbb{R}$, the simplest choice is the linear (affine) model, with $d = 2$ parameters:

$$\tilde{f}_{\boldsymbol{\theta}}(x) = \theta_1 + \theta_2\, x, \qquad \boldsymbol{\theta} = (\theta_1, \theta_2) \in \mathbb{R}^2.$$

Fitting $\boldsymbol{\theta}$ means making the residuals $y_i - \tilde{f}_{\boldsymbol{\theta}}(x_i)$ small (Fig. 1). This toy example already contains the three ingredients of the course: a model, a loss that measures the residuals, and a method that minimizes it. Since f is in general not linear, we need richer models, which is the purpose of neural networks.

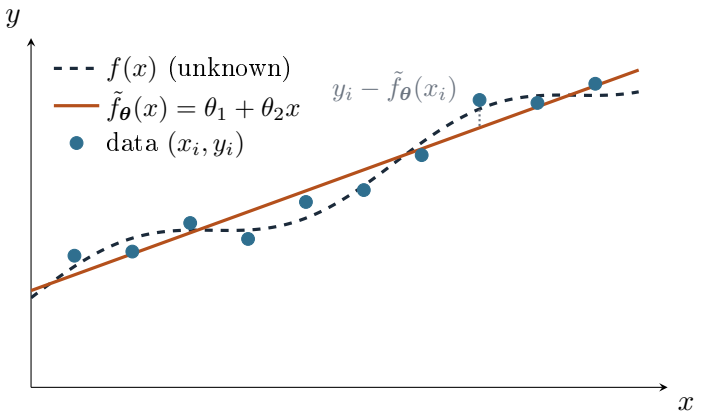


Figure 1: Linear model $\tilde{f}_{\boldsymbol{\theta}}$ fitted to samples of an unknown function f.

1.3 Notation

The following notation is used throughout.

Symbol	Meaning
n, m	input and output dimensions
$\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{y}_i\}_{i=1}^N$	dataset of N samples, $\boldsymbol{x}_i \in \mathbb{R}^n$, $\boldsymbol{y}_i \in \mathbb{R}^m$
$\tilde{f}_{\boldsymbol{\theta}}$, $\boldsymbol{\theta} \in \mathbb{R}^d$	model and its d parameters
L	number of layers (depth)
p_ℓ	number of neurons (width) of layer ℓ, with $p_0 = n$, $p_L = m$
$W_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}}$, $\boldsymbol{b}_\ell \in \mathbb{R}^{p_\ell}$	weights and biases of layer ℓ
σ_ℓ, h_ℓ	activation function and map of layer ℓ
$\boldsymbol{z}_\ell \in \mathbb{R}^{p_\ell}$	output of layer ℓ, with $\boldsymbol{z}_0 = \boldsymbol{x}$, $\boldsymbol{z}_L = \boldsymbol{y}$
$[\boldsymbol{z}]_j$	j-th component of a vector $\boldsymbol{z}$
$J(\hat{\boldsymbol{y}}, \boldsymbol{y})$	pointwise loss between a prediction $\hat{\boldsymbol{y}}$ and a target $\boldsymbol{y}$
$\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta})$	loss function (empirical risk) on the dataset $\mathcal{D}$
k, $\boldsymbol{\theta}_k$	iteration index and iterate of an optimization method
η	learning rate (step size)
I_k, B	mini-batch at iteration k and its size
$[\![1, N]\!]$	the integers $1, 2, \dots, N$

1.4 Outline

Section 2 builds the model $\tilde{f}_{\boldsymbol{\theta}}$, from the perceptron to the multilayer perceptron, and explains why it is a sensible choice. Section 3 shows how to fit $\boldsymbol{\theta}$: the loss function, the treatment of the dataset, the initialization, and gradient-based optimization methods, whose gradients are computed by automatic differentiation (AD). It ends with the training loop used in practice.

2 The model $\tilde{f}_{\boldsymbol{\theta}}$: neural networks

2.1 From the perceptron to the neural unit

Neural networks derive from the *perceptron*, introduced by Frank Rosenblatt in 1958 [1] as a simple model of a biological neuron.

Perceptron. Given an input $\boldsymbol{x} \in \mathbb{R}^n$, the model has weights $\boldsymbol{w} \in \mathbb{R}^n$ and a bias $b \in \mathbb{R}$. The perceptron first computes a weighted sum of its inputs, then applies a threshold:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{j=1}^{n} w_j x_j + b, \qquad y = \mathrm{sg}(z) = \begin{cases} 1, & \text{if } \boldsymbol{w}^\top \boldsymbol{x} + b > 0, \\ 0, & \text{otherwise,} \end{cases} \qquad \longleftarrow \text{activation}$$

where sg denotes the step function. As for a biological neuron, $y = 1$ means that the information passes and $y = 0$ that it does not. Geometrically, the perceptron separates the input space into two classes by a *linear decision boundary*, the hyperplane $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ (Fig. 2, right).

Neural unit. Replacing the step function by a general (nonlinear) *activation function* σ gives the elementary brick of all networks, the *neural unit*:

$$\boxed{y = \sigma\big(\boldsymbol{w}^\top \boldsymbol{x} + b\big) = h(\boldsymbol{x})}$$

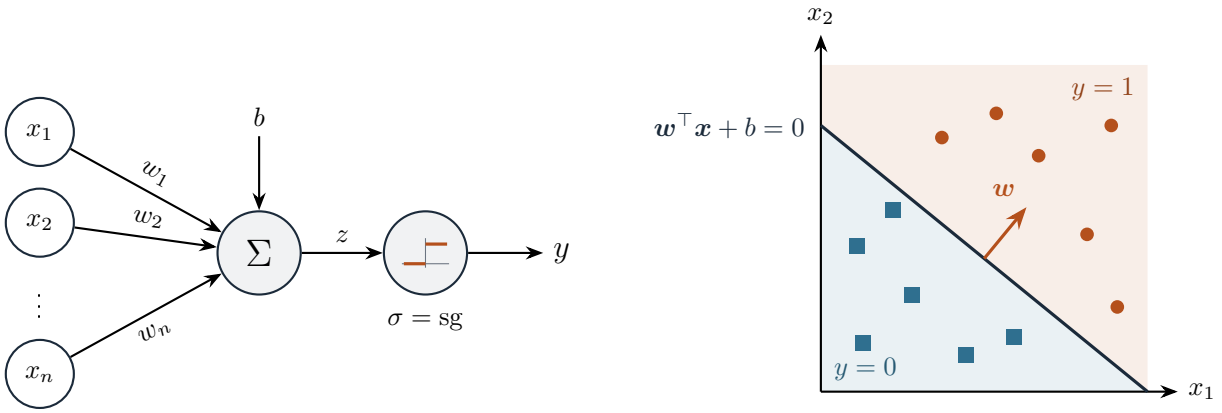
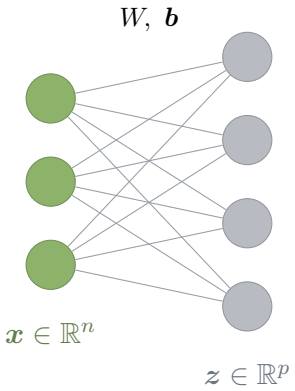


Figure 2: Left: Rosenblatt's perceptron $y = \mathrm{sg}(\boldsymbol{w}^\top \boldsymbol{x} + b)$, i.e. the neural unit with $\sigma = \mathrm{sg}$ (step function). Right: in the input plane ($n = 2$), the decision boundary $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ is a line of normal $\boldsymbol{w}$; the perceptron outputs $y = 1$ on the side $\boldsymbol{w}$ points to ($\boldsymbol{w}^\top \boldsymbol{x} + b > 0$), and $y = 0$ on the other side.

2.2 One layer of neurons

A neural network is a collection of interconnected nodes (neurons), organized in layers. Each layer processes its input, applies weights and biases, then an activation function, and passes the result to the next layer. A layer of p neurons acting on $\boldsymbol{x} \in \mathbb{R}^n$ reads as follows.



Neuron $j \in [\![1, p]\!]$ is a neural unit with its own weights $\boldsymbol{w}_j \in \mathbb{R}^n$ and bias $b_j \in \mathbb{R}$; it gives the j-th component of the output $\boldsymbol{z}$:

$$[\boldsymbol{z}]_j = \sigma\big(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j\big), \qquad j = 1, \dots, p.$$

Stacking the rows $\boldsymbol{w}_j^\top$ into a matrix W:

$$\implies \quad \boxed{\boldsymbol{z} = h(\boldsymbol{x}) = \sigma(W\boldsymbol{x} + \boldsymbol{b}) \in \mathbb{R}^p},$$

with $W \in \mathbb{R}^{p \times n}$ and $\boldsymbol{b} \in \mathbb{R}^p$.

1 layer $\qquad \boldsymbol{\theta} = (W, \boldsymbol{b}) \in \mathbb{R}^{p(n+1)}$.

2.3 Activation functions

The activation function $\sigma : \mathbb{R} \to \mathbb{R}$ is applied to each component of its argument. The most common choices are listed below and plotted in Fig. 3.

Name	$\sigma(z)$	Range
ReLU (Rectified Linear Unit)	$\max(0, z)$	$\mathbb{R}^+$
Sigmoid	$\dfrac{1}{1+e^{-z}}$	$]0, 1[$
tanh	$\dfrac{e^z - e^{-z}}{e^z + e^{-z}}$	$]-1, 1[$
Leaky ReLU [14]	$\max(\alpha z, z),\ \ \alpha \approx 10^{-2}$	$\mathbb{R}$
ELU (Exponential Linear Unit) [15]	z if $z > 0$, $\ \alpha(e^z - 1)$ otherwise, $\ \alpha = 1$	$]-\alpha, +\infty[$
Mish [16]	$z \tanh\big(\ln(1 + e^z)\big)$	$[\approx -0.31, +\infty[$

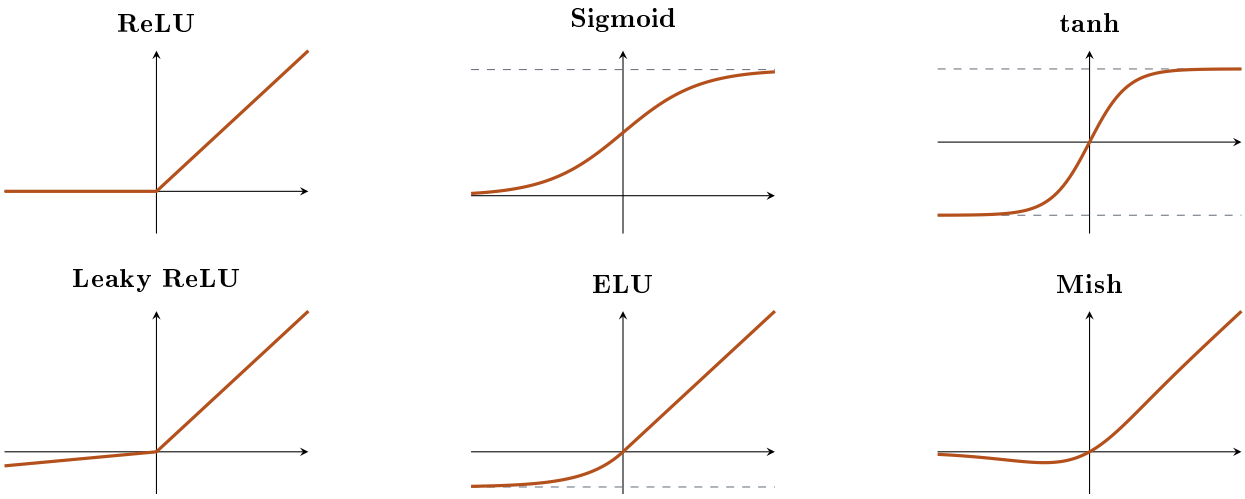


Figure 3: Activation functions (same axes on each row; Leaky ReLU drawn with $\alpha = 0.1$ for visibility).

Sigmoid and tanh saturate ($\sigma'(z) \to 0$ as $|z| \to \infty$), which slows down training in deep networks. ReLU does not saturate for $z > 0$, but its second derivative vanishes almost everywhere: smooth activations (tanh, Mish, ...) are therefore preferred for PINNs, which differentiate the network.

2.4 The multilayer perceptron (MLP)

To approximate $\boldsymbol{y} = f(\boldsymbol{x}) \in \mathbb{R}^m$, we now chain L layers. Layer $\ell \in [\![1, L]\!]$ has p_ℓ neurons and an activation function σ_ℓ; it is the map

$$h_\ell : \mathbb{R}^{p_{\ell-1}} \to \mathbb{R}^{p_\ell}, \qquad h_\ell(\boldsymbol{z}) = \sigma_\ell\big(W_\ell\, \boldsymbol{z} + \boldsymbol{b}_\ell\big), \qquad W_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}}, \quad \boldsymbol{b}_\ell \in \mathbb{R}^{p_\ell},$$

with the conventions $p_0 = n$ (input dimension) and $p_L = m$ (output dimension). The layers $\ell = 1, \dots, L-1$ are the *hidden layers*; p_ℓ is the *width* of layer ℓ and L the *depth* of the network (Fig. 4).

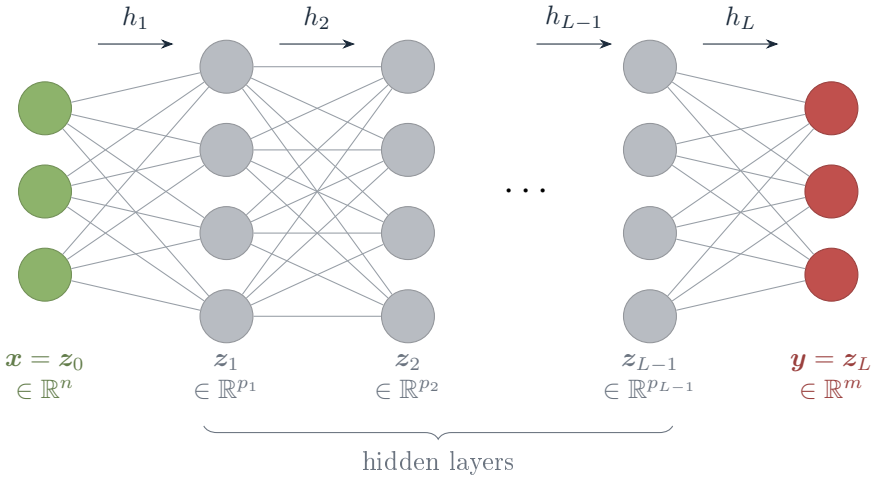


Figure 4: A vanilla MLP of depth L: input layer (green), hidden layers (grey), output layer (red). Every neuron of a layer is connected to every neuron of the next one.

Starting from $\boldsymbol{z}_0 = \boldsymbol{x}$, the layers are applied one after the other:

$$\begin{aligned}
\boldsymbol{z}_1 &= h_1(\boldsymbol{x}) = \sigma_1\big(W_1\boldsymbol{x} + \boldsymbol{b}_1\big), && W_1 \in \mathbb{R}^{p_1 \times n}, \quad \boldsymbol{b}_1 \in \mathbb{R}^{p_1}, \\
\boldsymbol{z}_2 &= h_2(\boldsymbol{z}_1) = \sigma_2\big(W_2\boldsymbol{z}_1 + \boldsymbol{b}_2\big), && W_2 \in \mathbb{R}^{p_2 \times p_1}, \quad \boldsymbol{b}_2 \in \mathbb{R}^{p_2}, \\
&\vdots \\
\boldsymbol{y} = \boldsymbol{z}_L &= h_L(\boldsymbol{z}_{L-1}) = \sigma_L\big(W_L\boldsymbol{z}_{L-1} + \boldsymbol{b}_L\big), && W_L \in \mathbb{R}^{m \times p_{L-1}}, \quad \boldsymbol{b}_L \in \mathbb{R}^{m}.
\end{aligned}$$

The network is therefore a **composition of layers**:

$$\boxed{\boldsymbol{y} = \big(h_L \circ h_{L-1} \circ \cdots \circ h_1\big)(\boldsymbol{x}) = \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}), \qquad \boldsymbol{\theta} = \big\{W_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}},\ \boldsymbol{b}_\ell \in \mathbb{R}^{p_\ell}\big\}_{\ell=1}^{L}}$$

Hyperparameters. The following quantities are chosen by the user and are not learned:

- the number of layers L;
- for each layer ℓ, the number of neurons p_ℓ and the activation function σ_ℓ.

Last layer. The activation of the output layer depends on the nature of $\boldsymbol{y}$:

- regression: no activation ($\sigma_L = \mathrm{id}$), or tanh to normalize the output in $]-1, 1[$;
- classification: the output must be a probability (sigmoid for two classes, a normalized probability vector for C classes, see Section 3.1).

Names. This architecture appears in the literature under several names:

- MLP (Multilayer Perceptron),
- FNN (Feedforward NN),
- FCN (Fully Connected Network),
- DNN (Dense Neural Network),
- Vanilla NN $\rightsquigarrow$ basic, compared to CNN / RNN; "vanilla MLP" also means without add-ons such as dropout, batch normalization or Fourier features.

2.5 Number of parameters

Layer ℓ has $p_\ell \times p_{\ell-1}$ weights and p_ℓ biases, i.e. $p_\ell\,(p_{\ell-1}+1)$ parameters. Gathering all of them in one vector $\boldsymbol{\theta} \in \mathbb{R}^d$ gives

$$\boxed{d = \sum_{\ell=1}^{L} p_\ell\,(p_{\ell-1}+1)}$$

Layer ℓ	Shape of W_ℓ	Weights	Biases	Parameters
1 (input $\to$ hidden)	$p_1 \times n$	$p_1 n$	p_1	$p_1(n+1)$
ℓ (hidden $\to$ hidden)	$p_\ell \times p_{\ell-1}$	$p_\ell p_{\ell-1}$	p_ℓ	$p_\ell(p_{\ell-1}+1)$
L (hidden $\to$ output)	$m \times p_{L-1}$	$m\,p_{L-1}$	m	$m(p_{L-1}+1)$
Total				$d = \sum_\ell p_\ell(p_{\ell-1}+1)$

Example. A typical PINN for $u(x,t)$ has $n = 2$ inputs, three hidden layers of width 20 and $m = 1$ output, so $(p_0, p_1, p_2, p_3, p_4) = (2, 20, 20, 20, 1)$ and

$$d = 20 \cdot 3 + 20 \cdot 21 + 20 \cdot 21 + 1 \cdot 21 = 60 + 420 + 420 + 21 = 921.$$

2.6 Why MLPs? The universal approximation theorem

The choice of the MLP is justified by approximation theory: a network with a single hidden layer can already approximate any continuous function, provided it is wide enough [2, 3, 17].

Theorem 1 (Universal approximation)**.** *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and* not a polynomial*, and let $K \subset \mathbb{R}^n$ be compact. For every continuous function $f : K \to \mathbb{R}^m$ and every $\varepsilon > 0$, there exist a width $p \in \mathbb{N}$ and parameters $W \in \mathbb{R}^{p\times n}$, $\boldsymbol{b} \in \mathbb{R}^p$, $V \in \mathbb{R}^{m\times p}$ such that*

$$\sup_{\boldsymbol{x}\in K} \big\| f(\boldsymbol{x}) - V\sigma(W\boldsymbol{x} + \boldsymbol{b}) \big\| < \varepsilon.$$

Comments.

- *Nonlinearity is essential.* The assumption "σ not a polynomial" rules out in particular $\sigma = \mathrm{id}$: without activation functions, every layer is affine, and so is their composition, $\boldsymbol{y} = W_L(\cdots(W_1\boldsymbol{x} + \boldsymbol{b}_1)\cdots) + \boldsymbol{b}_L = W\boldsymbol{x} + \boldsymbol{b}$, whatever the depth. All the activations of Fig. 3 satisfy the assumption.
- *Existence only.* The theorem guarantees that good parameters exist, but says nothing about the width p needed, nor about how to find these parameters (Section 3), nor about how well they generalize to new inputs.
- *Depth.* In practice, deeper networks (several hidden layers) reach a given accuracy with far fewer parameters than a single very wide layer, which is why MLPs with a few hidden layers are used.

3 Training: finding an optimal $\boldsymbol{\theta}$

3.1 Formulation

Finding $\boldsymbol{\theta}$ is a high-dimensional optimization problem: we minimize a **loss function** (also called *cost function* or *empirical risk*) $\mathcal{L}_\mathcal{D} : \mathbb{R}^d \to \mathbb{R}^+$, which measures the misfit between the model and the data:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \mathcal{L}_\mathcal{D}(\boldsymbol{\theta}), \qquad (d \text{ quite big}).$$

In general, $\mathcal{L}_\mathcal{D}$ is the average over the dataset of a pointwise loss $J(\hat{\boldsymbol{y}}, \boldsymbol{y})$, which compares a prediction $\hat{\boldsymbol{y}}$ with the target $\boldsymbol{y}$:

$$\mathcal{L}_\mathcal{D}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} J\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i\big).$$

The choice of J depends on the task.

Regression (the setting of this course, and of PINNs): $\boldsymbol{y} \in \mathbb{R}^m$ is a continuous quantity. The loss is the **Mean Squared Error (MSE)**:

$$\mathcal{L}_{\mathrm{MSE}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \big\|\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i\big\|_2^2, \qquad \mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{y}_i\}_{i=1}^N.$$

For $m = 1$ this is $\frac{1}{N}\sum_i (\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i)^2$.

Other tasks, other losses. The same network can be used for classification or other tasks; what changes is the output layer and the way the function to optimize is written.

Multiclass classification (C classes). Each label is *one-hot encoded*: if $\boldsymbol{x}_i$ belongs to class c_i, then $\boldsymbol{y}_i = \boldsymbol{e}_{c_i} \in \{0,1\}^C$ (the c_i-th vector of the canonical basis). The output of the last layer, $\boldsymbol{a} = W_L \boldsymbol{z}_{L-1} + \boldsymbol{b}_L \in \mathbb{R}^C$, is normalized by the *softmax* $\hat{y}_c = e^{a_c} / \sum_{j=1}^{C} e^{a_j}$, so that the output lies in the probability simplex

$$\hat{\boldsymbol{y}}_i = \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) \in \Delta_C = \left\{ \boldsymbol{q} \in]0,1[^C : \sum_{c=1}^{C} q_c = 1 \right\}, \qquad \hat{y}_{i,c} \approx \mathbb{P}(\text{class } c \mid \boldsymbol{x}_i).$$

The loss is the **categorical cross-entropy**:

$$\mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \ln \hat{y}_{i,c} = -\frac{1}{N} \sum_{i=1}^{N} \ln \hat{y}_{i,c_i}.$$

Since $\boldsymbol{y}_i$ is one-hot, only the probability given to the true class counts: the loss is 0 if $\hat{y}_{i,c_i} = 1$ and tends to $+\infty$ if $\hat{y}_{i,c_i} \to 0$.

Both losses are negative log-likelihoods: minimizing $\mathcal{L}_{\text{CE}}$ maximizes $\prod_i \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{y}_i \mid \boldsymbol{x}_i)$ for a categorical model, and minimizing $\mathcal{L}_{\text{MSE}}$ does the same for a Gaussian noise model $\boldsymbol{y}_i = f(\boldsymbol{x}_i) + \boldsymbol{\varepsilon}_i$, $\boldsymbol{\varepsilon}_i \sim \mathcal{N}(0, s^2 I)$ [13].

Task	Output $\boldsymbol{y}$	Last activation σ_L	Loss
Regression	$\mathbb{R}^m$	none (identity), or tanh	$\mathcal{L}_{\text{MSE}}$
Binary classification	$\{0,1\}$	sigmoid	binary cross-entropy
Multiclass classification	one-hot in $\{0,1\}^C$	softmax	$\mathcal{L}_{\text{CE}}$

3.2 Dataset: normalization and splitting

The dataset is first **normalized** (e.g. each input and output component rescaled to $[-1,1]$, or to zero mean and unit variance, using statistics of the training set only). It is then split at random into three disjoint subsets, each with its own role (Fig. 5).

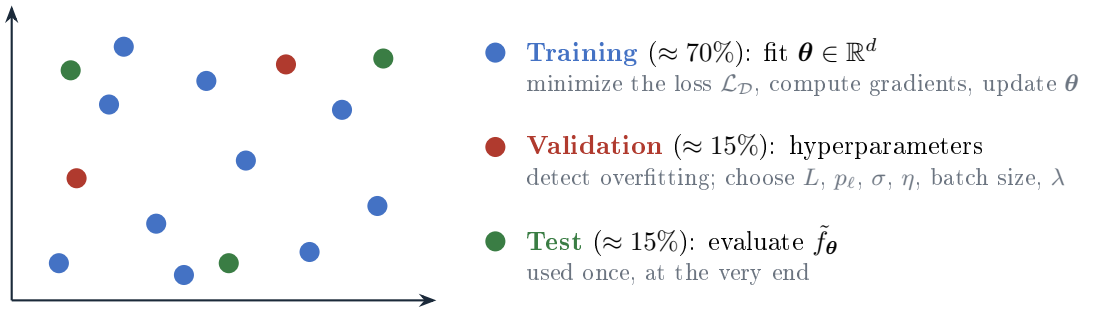


Figure 5: Random split of a normalized dataset $\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{y}_i\}$ into training, validation and test sets.

- **Training set**: the only data the loss $\mathcal{L}_{\mathcal{D}}$ (and its gradient) is computed on. It is used to fit the parameters $\boldsymbol{\theta}$ (weights and biases).
- **Validation set**: never used to update $\boldsymbol{\theta}$. It is evaluated during training to
  - detect *overfitting*: the training loss keeps decreasing while the validation loss increases, i.e. the model memorizes the training points instead of learning f (Fig. 6); training can be stopped at the minimum of the validation loss (*early stopping*);
  - fit the *hyperparameters*, i.e. everything that is not in $\boldsymbol{\theta}$: the architecture (number of layers L, widths p_ℓ, activation functions), the learning rate η, the batch size, the regularization weight λ, ...

- **Test set**: used only once, at the end, once all choices are frozen, to assess how the model really behaves on unseen data. Since neither $\boldsymbol{\theta}$ nor the hyperparameters were tuned on it, the test error is an unbiased estimate of the generalization error.

The trained model is then used during *inference*: evaluation on new inputs, without gradients.

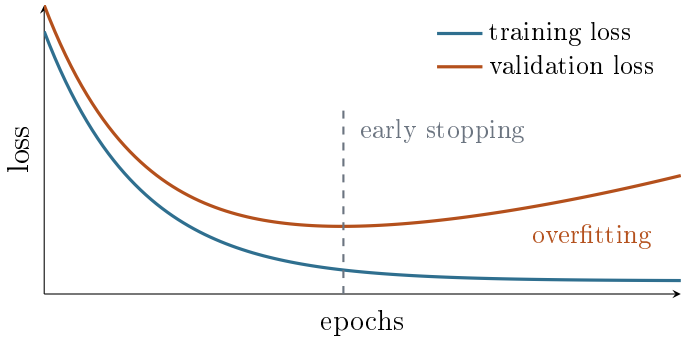


Figure 6: Overfitting, as seen on the validation set.

3.3 Initialization of $\boldsymbol{\theta}_0$

All the optimization methods below are iterative: they start from an initial guess $\boldsymbol{\theta}_0$, whose choice matters for deep networks.

Why not $\boldsymbol{\theta}_0 = 0$? If all the weights of a layer are equal, all its neurons compute the same output and receive the same gradient: they stay identical during the whole training (*symmetry*). The weights are therefore drawn at random, and the biases set to $\boldsymbol{b}_\ell = 0$.

Principle. Draw the entries of W_ℓ i.i.d., with zero mean and variance s_ℓ^2. The variance s_ℓ^2 is chosen so that the signals (forward pass) and the gradients (backward pass) keep the same order of magnitude from one layer to the next; otherwise they vanish or explode with the depth L. With $\boldsymbol{a}_\ell = W_\ell \boldsymbol{z}_{\ell-1}$ (pre-activation of layer ℓ),

$$\mathrm{Var}\left([\boldsymbol{a}_\ell]_j\right) = p_{\ell-1}\, s_\ell^2\, \mathbb{E}\big[[\boldsymbol{z}_{\ell-1}]_i^2\big],$$

which leads to a variance inversely proportional to the number of incoming neurons $p_{\ell-1}$ (*fan-in*).

Activation	Initialization	Normal version	Uniform version
tanh, sigmoid	Glorot (aka Xavier) [6]	$\mathcal{N}\left(0, \dfrac{2}{p_{\ell-1}+p_\ell}\right)$	$\mathcal{U}\left[-\sqrt{\dfrac{6}{p_{\ell-1}+p_\ell}}, \sqrt{\dfrac{6}{p_{\ell-1}+p_\ell}}\right]$
ReLU	He (aka Kaiming) [7]	$\mathcal{N}\left(0, \dfrac{2}{p_{\ell-1}}\right)$	$\mathcal{U}\left[-\sqrt{\dfrac{6}{p_{\ell-1}}}, \sqrt{\dfrac{6}{p_{\ell-1}}}\right]$

- **Glorot**: compromise between the forward constraint ($p_{\ell-1}s_\ell^2 = 1$) and the backward one ($p_\ell s_\ell^2 = 1$), hence the harmonic mean $2/(p_{\ell-1}+p_\ell)$. Suited to symmetric, saturating activations; the standard choice for PINNs with tanh.
- **He**: ReLU sets half of its inputs to zero, $\mathbb{E}[\max(0,a)^2] = \frac{1}{2}\mathbb{E}[a^2]$, so the variance is doubled: $2/p_{\ell-1}$.

In PyTorch: `torch.nn.init.xavier_normal_(W)` and `torch.nn.init.kaiming_normal_(W)`.

K. He (now Associate Professor at MIT) is also the first author of *Deep Residual Learning for Image Recognition* [8].

3.4 Overview of gradient-based optimization methods

Principle. Starting from $\boldsymbol{\theta}_0$, we build a sequence $(\boldsymbol{\theta}_k)_{k\in\mathbb{N}}$ that decreases $\mathcal{L}$, using local information on $\mathcal{L}$ around $\boldsymbol{\theta}_k$, given by its Taylor expansion:

$$\mathcal{L}(\boldsymbol{\theta}_k + \boldsymbol{\delta}) = \mathcal{L}(\boldsymbol{\theta}_k) + \nabla\mathcal{L}(\boldsymbol{\theta}_k)^\top \boldsymbol{\delta} + \tfrac{1}{2}\,\boldsymbol{\delta}^\top \nabla^2\mathcal{L}(\boldsymbol{\theta}_k)\,\boldsymbol{\delta} + o(\|\boldsymbol{\delta}\|^2).$$

The gradient $\nabla\mathcal{L}(\boldsymbol{\theta}) \in \mathbb{R}^d$ is computed exactly by automatic differentiation (backpropagation), at a cost comparable to one evaluation of the network. The methods differ by how much of this expansion they use.

First-order methods use only the gradient (the first derivative). The simplest is **gradient descent**, $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\nabla\mathcal{L}(\boldsymbol{\theta}_k)$; its stochastic and adaptive variants (SGD, Momentum, RMSProp, Adam) are the workhorses of deep learning (Section 3.5).

Second-order methods also use the Hessian $\nabla^2\mathcal{L}(\boldsymbol{\theta}_k) \in \mathbb{R}^{d\times d}$. The reference method, the "Rolls-Royce" of optimization, is **Newton's method**, which minimizes the quadratic model in $\boldsymbol{\delta}$:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \left[\nabla^2\mathcal{L}(\boldsymbol{\theta}_k)\right]^{-1}\nabla\mathcal{L}(\boldsymbol{\theta}_k).$$

Since the Hessian is expensive, the classical methods are **quasi-Newton** methods (BFGS, L-BFGS), which approximate it from successive gradients only (Section 3.7).

Family	Example	Memory	Advantages	Drawbacks
First order	GD, SGD, Adam	$O(d)$	cheap iterations; scales to $d \sim 10^6$–10^9; works with noisy (mini-batch) gradients	slow (at best linear) convergence; sensitive to conditioning and to η
Second order	Newton	$O(d^2)$ storage, $O(d^3)$ solve	quadratic local convergence; no step size to tune	intractable for large d; Hessian may be indefinite (non-convex $\mathcal{L}$): attracted by saddle points
Quasi-second order	BFGS, L-BFGS	$O(d^2)$, L-BFGS: $O(rd)$	superlinear convergence without Hessian; very accurate final minimum	needs full-batch, deterministic gradients and a line search; can stall in non-convex landscapes

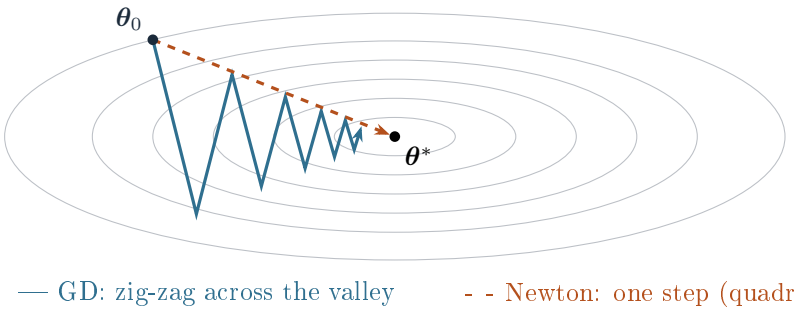


Figure 7: On the ill-conditioned quadratic $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1^2 + 10\,\theta_2^2)$, GD ($\eta = 0.18$) zig-zags while Newton reaches $\boldsymbol{\theta}^*$ in one step.

In deep learning, d is large and the gradient is only affordable on a few samples at a time, so first-order methods are the standard. Second-order methods become attractive when d is small, as for PINNs.

3.5 First-order methods

First-order methods only use the gradient. We present them in the order in which each one corrects a weakness of the previous one: gradient descent (GD), then Momentum (faster convergence), RMSProp (one adaptive step per parameter), and finally Adam, which combines both. In this section, the gradient is computed on the whole training set (*full batch*) and denoted

$$\boldsymbol{g}_k = \nabla \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}_k);$$

Section 3.9 explains how it is replaced in practice by a cheaper, noisy estimate computed on a mini-batch.

3.5.1 Gradient descent with a fixed learning rate

Keeping only the linear term of the Taylor expansion, the steepest descent direction is $-\nabla\mathcal{L}(\boldsymbol{\theta}_k)$. Gradient descent (GD) moves a fixed step in that direction:

$$\begin{cases} \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\,\nabla\mathcal{L}(\boldsymbol{\theta}_k), & \eta = \text{learning rate / step size } (10^{-3} \text{ typically}), \\ \text{start from } \boldsymbol{\theta}_0. \end{cases}$$

The choice of η is critical. On the model problem $\mathcal{L}(\theta) = \theta^2$, GD gives $\theta_{k+1} = (1-2\eta)\,\theta_k$: it converges if and only if $0 < \eta < 1$, slowly if η is small, and it diverges if η is too large (Fig. 8).

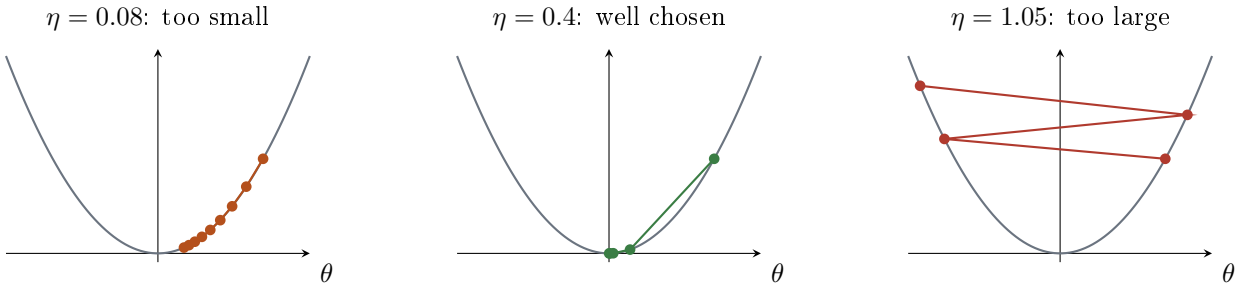


Figure 8: Gradient descent on $\mathcal{L}(\theta) = \theta^2$ from $\theta_0 = 1.8$, for three learning rates.

Why go further than GD? Plain GD has two weaknesses, which motivate the next three methods:

i) *slow convergence on ill-conditioned problems*: when the loss is much steeper in some directions than in others (a narrow valley), a single η cannot suit all directions and GD zig-zags across the valley (Fig. 7) $\rightarrow$ Momentum;

ii) *high sensitivity to* η: the same step is used for all d parameters, whose gradients may have very different scales $\rightarrow$ RMSProp;

and Adam combines both corrections (Fig. 9).

3.5.2 Momentum

Drawback i): suboptimal rate of convergence. On ill-conditioned problems, GD zig-zags (Fig. 7). Inertial optimizers (**Momentum**) damp these oscillations by averaging the successive gradients. $(\boldsymbol{v}_k)_{k\in\mathbb{N}}$ represents an exponential moving average of the gradients:

$$\begin{cases} \boldsymbol{v}_{k+1} = \beta\boldsymbol{v}_k + \eta\,\boldsymbol{g}_k, & \boldsymbol{v}_0 = 0_{\mathbb{R}^d}, \\ \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{v}_{k+1}. \end{cases}$$

$\beta \in]0,1[$ measures the trade-off between the actual gradient and the previous ones. Components of the gradient that keep the same sign (along the valley) accumulate, while those that alternate (across the valley) cancel out (Fig. 9).

3.5.3 RMSProp

Drawback ii): high sensitivity to the choice of η. A single learning rate for all d parameters is a poor compromise when the gradient components have very different scales. **RMSProp** (Root Mean Squared Propagation) [4] is an adaptive method: it rescales each component by a running estimate of its magnitude, which also deals with vanishing / exploding gradients.

$$\begin{cases} \boldsymbol{s}_{k+1} = \beta \boldsymbol{s}_k + (1-\beta)\, \boldsymbol{g}_k^2, & \boldsymbol{s}_0 = 0_{\mathbb{R}^d}, \\ \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \dfrac{\boldsymbol{g}_k}{\varepsilon_a + \sqrt{\boldsymbol{s}_{k+1}}}, & \varepsilon_a > 0 \text{ small } (\approx 10^{-8}). \end{cases}$$

$(\cdot)^2$, $\frac{\cdot}{\cdot}$, $\sqrt{\cdot} \Longrightarrow$ **component-wise!**

3.5.4 Adam

Adam (Adaptive Moment Estimation) [5] combines i) and ii): a momentum on the gradient (first moment $\boldsymbol{m}_k$, mean of gradients) and an RMSProp scaling (second moment $\boldsymbol{v}_k$, uncentered variance of gradients). Since $\boldsymbol{m}_0 = \boldsymbol{v}_0 = 0$, both averages are biased towards zero during the first iterations; dividing by $1-\beta_1^k$ and $1-\beta_2^k$ corrects this bias. It is the default optimizer in deep learning, and the usual first stage of PINN training.

Algorithm 1: Adam [5]

Input: initial parameters $\boldsymbol{\theta}_0$; step size $\alpha = 10^{-3}$; decay rates $\beta_1 = 0.9$, $\beta_2 = 0.999$; $\varepsilon = 10^{-8}$; number of iterations K

$\boldsymbol{m}_0 \leftarrow 0$, $\boldsymbol{v}_0 \leftarrow 0$;

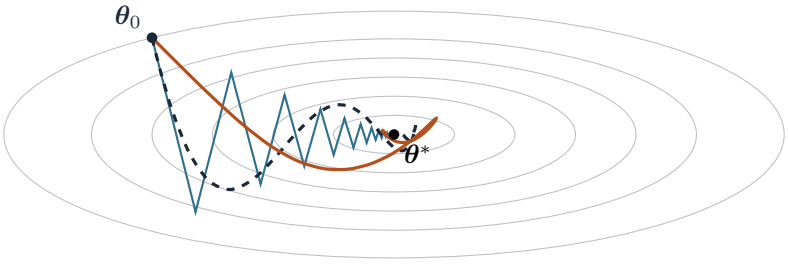
for $k = 1$ **to** K **do**

- $\boldsymbol{g}_k \leftarrow \nabla \mathcal{L}(\boldsymbol{\theta}_{k-1})$; `# gradient by backpropagation`
- $\boldsymbol{m}_k \leftarrow \beta_1 \boldsymbol{m}_{k-1} + (1-\beta_1)\, \boldsymbol{g}_k$; `# 1st moment`
- $\boldsymbol{v}_k \leftarrow \beta_2 \boldsymbol{v}_{k-1} + (1-\beta_2)\, \boldsymbol{g}_k^2$; `# 2nd moment`
- $\hat{\boldsymbol{m}}_k \leftarrow \boldsymbol{m}_k/(1-\beta_1^k)$, $\hat{\boldsymbol{v}}_k \leftarrow \boldsymbol{v}_k/(1-\beta_2^k)$; `# bias correction`
- $\boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_{k-1} - \alpha\, \hat{\boldsymbol{m}}_k / \left(\varepsilon + \sqrt{\hat{\boldsymbol{v}}_k}\right)$; `# component-wise`

Output: $\boldsymbol{\theta}_K$

Adam is sensitive to α, β_1, β_2. Kingma & Ba, ICLR 2015 [5]: one of the most cited papers in machine learning.

Figure 9 compares the three methods on the ill-conditioned quadratic of Fig. 7: GD zig-zags, Momentum damps the oscillations, and Adam, thanks to its per-component scaling, moves at the same speed in both directions.



— GD ($\eta = 0.18$): zig-zag
- - Momentum ($\eta = 0.03$, $\beta = 0.8$): damped
— Adam ($\alpha = 0.25$): rescaled steps

Figure 9: Trajectories of GD, Momentum and Adam on $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1^2 + 10\,\theta_2^2)$. Adam's per-component scaling makes the first steps equal in both directions (diagonal start).

3.6 The learning rate and its schedule

The learning rate η (α for Adam) is the most important hyperparameter of a first-order method (Fig. 8). A fixed value is a compromise: large steps make fast progress at the beginning, but prevent convergence to an accurate minimum at the end. A *learning-rate schedule* therefore decreases η during training. It is usually expressed as a function of the *epoch*, one pass over the training set; with full-batch gradients, as in this section, one epoch is exactly one iteration, so $\eta = \eta_k$ (Fig. 10). With η_0 the initial value and K the total number of epochs, common choices are:

- **step decay**: $\eta_k = \eta_0\,\gamma^{\lfloor k/s \rfloor}$, i.e. η is multiplied by $\gamma \in]0,1[$ every s epochs;
- **exponential decay**: $\eta_k = \eta_0\,\gamma^{k}$, with γ close to 1;
- **cosine annealing**: $\eta_k = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\big(1 + \cos(\pi k/K)\big)$.

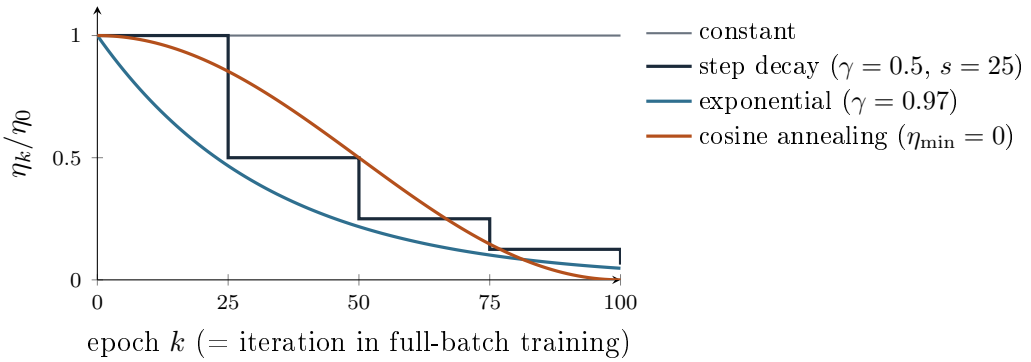


Figure 10: Common learning-rate schedules: learning rate as a function of the epoch, over $K = 100$ epochs.

A short *warm-up*, during which η increases linearly from 0 to η_0, is often added to stabilize the first epochs. Adam combined with a decaying schedule is a robust default. In PyTorch, schedules are provided by `torch.optim.lr_scheduler` (`StepLR`, `ExponentialLR`, `CosineAnnealingLR`).

3.7 Second-order methods

3.7.1 Newton's method

Newton's method minimizes, at each iteration, the quadratic model of $\mathcal{L}$ given by the second-order Taylor expansion around $\boldsymbol{\theta}_k$. Setting the gradient of this model with respect to $\boldsymbol{\delta}$ to zero gives $\nabla^2\mathcal{L}(\boldsymbol{\theta}_k)\,\boldsymbol{\delta} = -\nabla\mathcal{L}(\boldsymbol{\theta}_k)$, hence

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \big[\nabla^2\mathcal{L}(\boldsymbol{\theta}_k)\big]^{-1}\nabla\mathcal{L}(\boldsymbol{\theta}_k).$$

In one dimension, this is $\theta_{k+1} = \theta_k - \mathcal{L}'(\theta_k)/\mathcal{L}''(\theta_k)$: the next iterate is the minimum of the parabola that matches $\mathcal{L}$, $\mathcal{L}'$ and $\mathcal{L}''$ at θ_k (Fig. 11). Near a minimum where the Hessian is positive definite, Newton's method converges *quadratically*: the number of correct digits roughly doubles at each iteration.

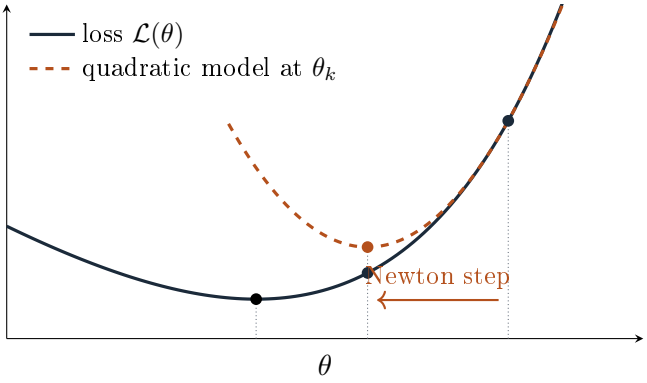


Figure 11: One Newton step on $\mathcal{L}(\theta) = e^\theta - 2\theta$ from $\theta_k = 2$: θ_{k+1} is the minimizer of the quadratic model (dashed), which matches $\mathcal{L}$ up to second order at θ_k.

Is there still a learning rate? In its pure form, Newton's method has *no learning rate*: the Hessian sets both the direction and the length of the step (the step length is 1), and on a quadratic loss the minimum is reached in one iteration (Fig. 7). Far from the minimum, or when the Hessian is not positive definite (non-convex $\mathcal{L}$), the full step may increase the loss. Practical (quasi-)Newton methods therefore compute a step length η_k *automatically* at each iteration, by a line search. A standard choice is to accept η_k when the *Wolfe conditions* hold, with $\boldsymbol{p}_k$ the search direction and $0 < c_1 < c_2 < 1$ (typically $c_1 = 10^{-4}$, $c_2 = 0.9$):

$$\mathcal{L}(\boldsymbol{\theta}_k + \eta_k \boldsymbol{p}_k) \le \mathcal{L}(\boldsymbol{\theta}_k) + c_1\, \eta_k\, \nabla\mathcal{L}(\boldsymbol{\theta}_k)^\top \boldsymbol{p}_k, \qquad \nabla\mathcal{L}(\boldsymbol{\theta}_k + \eta_k \boldsymbol{p}_k)^\top \boldsymbol{p}_k \ge c_2\, \nabla\mathcal{L}(\boldsymbol{\theta}_k)^\top \boldsymbol{p}_k.$$

The first condition guarantees a sufficient decrease of the loss, the second rules out too small steps. So there is no learning rate to tune by hand, but other (milder) hyperparameters remain: line-search constants, memory size, damping.

Cost. Each Newton iteration requires the $d \times d$ Hessian ($O(d^2)$ memory) and the solution of a linear system ($O(d^3)$ operations). This is affordable for small networks only, which motivates the approximations below.

3.7.2 Gauss–Newton and Levenberg–Marquardt

For a least-squares loss such as the MSE, the Hessian has a simple approximation. Write $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\|\boldsymbol{r}(\boldsymbol{\theta})\|^2$, with $\boldsymbol{r}(\boldsymbol{\theta})$ the vector of all residuals $\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - \boldsymbol{y}_i$ and $\mathsf{J} = \partial \boldsymbol{r}/\partial \boldsymbol{\theta}$ its Jacobian. Then $\nabla\mathcal{L} = \frac{2}{N}\mathsf{J}^\top \boldsymbol{r}$ and $\nabla^2\mathcal{L} = \frac{2}{N}\big(\mathsf{J}^\top\mathsf{J} + \sum_j r_j \nabla^2 r_j\big)$. Dropping the second term, which is small when the residuals are small, gives the **Gauss–Newton** method; adding a damping $\mu_k > 0$ gives **Levenberg–Marquardt**:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \big(\mathsf{J}^\top\mathsf{J}\big)^{-1}\mathsf{J}^\top \boldsymbol{r}, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \big(\mathsf{J}^\top\mathsf{J} + \mu_k I\big)^{-1}\mathsf{J}^\top \boldsymbol{r}.$$

Only first derivatives are needed, and $\mathsf{J}^\top\mathsf{J}$ is always positive semi-definite.

3.7.3 Quasi-Newton methods: BFGS and L-BFGS

Quasi-Newton methods keep most of Newton's speed at the cost of a first-order method, by replacing $[\nabla^2\mathcal{L}(\boldsymbol{\theta}_k)]^{-1}$ by an approximation H_k built from successive gradients only:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k\, \boldsymbol{p}_k, \qquad \boldsymbol{p}_k = -H_k \nabla\mathcal{L}(\boldsymbol{\theta}_k),$$

with η_k given by the line search above. With $\boldsymbol{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k$ and the gradient differences $\boldsymbol{q}_k = \nabla\mathcal{L}(\boldsymbol{\theta}_{k+1}) - \nabla\mathcal{L}(\boldsymbol{\theta}_k)$, **BFGS** enforces the secant condition $H_{k+1}\boldsymbol{q}_k = \boldsymbol{s}_k$ through the rank-two update

$$H_{k+1} = \big(I - \rho_k \boldsymbol{s}_k \boldsymbol{q}_k^\top\big) H_k \big(I - \rho_k \boldsymbol{q}_k \boldsymbol{s}_k^\top\big) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^\top, \qquad \rho_k = \frac{1}{\boldsymbol{q}_k^\top \boldsymbol{s}_k}.$$

L-BFGS (limited memory) never forms H_k: it only stores the last r pairs $(\boldsymbol{s}_k, \boldsymbol{q}_k)$, typically $r \approx 10$–50 [11, 12]. L-BFGS is the classical second-order method in machine learning. For PINNs, the standard strategy is to run Adam first, then switch to L-BFGS to refine the minimum.

3.7.4 Recent advances

Second-order methods are an active research topic for PINNs, where reaching a very small loss is essential. Among recent developments:

- **self-scaled quasi-Newton methods** (SSBFGS, SSBroyden) [18]. Before each BFGS update, H_k is rescaled to match the curvature observed along the last step,

$$H_k \leftarrow \tau_k H_k, \qquad \tau_k = \frac{\boldsymbol{s}_k^\top \boldsymbol{q}_k}{\boldsymbol{q}_k^\top H_k \, \boldsymbol{q}_k};$$

  on several PINN benchmarks, these methods were reported to improve the accuracy by orders of magnitude over Adam and L-BFGS [19];

- **natural gradient methods**, which precondition the gradient by the geometry of the model in function space,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \, G(\boldsymbol{\theta}_k)^\dagger \nabla \mathcal{L}(\boldsymbol{\theta}_k), \qquad G_{ij}(\boldsymbol{\theta}) = \langle \partial_{\theta_i} \tilde{f}_{\boldsymbol{\theta}}, \partial_{\theta_j} \tilde{f}_{\boldsymbol{\theta}} \rangle,$$

  where G is a Gram matrix and $\dagger$ the pseudo-inverse: energy natural gradient [20], Gauss–Newton natural gradient [21] and its scalable dual formulation [22].

When to use them. Second-order methods are affordable when d is small. This is the case for the small networks typical of PINNs: with $d \approx 10^3$ (e.g. $d = 921$ in the example of Section 2), the Hessian has about 10^6 entries, which is cheap to store and factorize. For large networks ($d \gtrsim 10^6$) and large datasets, first-order methods remain the only practical choice.

3.8 A worked example: GD, Adam and L-BFGS

To compare the methods, we fit $f(x) = \sin(2\pi x)$ on $[-1, 1]$ from $N = 256$ random samples, with an MLP $1 \to 20 \to 20 \to 1$ (tanh hidden layers, $d = 481$ parameters), Glorot initialization and the MSE loss, using full-batch gradients (one iteration = one epoch). All three methods start from the same $\boldsymbol{\theta}_0$ and run for 3000 iterations:

- GD with $\eta = 0.3$ (larger values diverge);
- Adam (Algorithm 1) with $\alpha = 10^{-2}$;
- L-BFGS with memory $r = 20$ and a Wolfe line search.

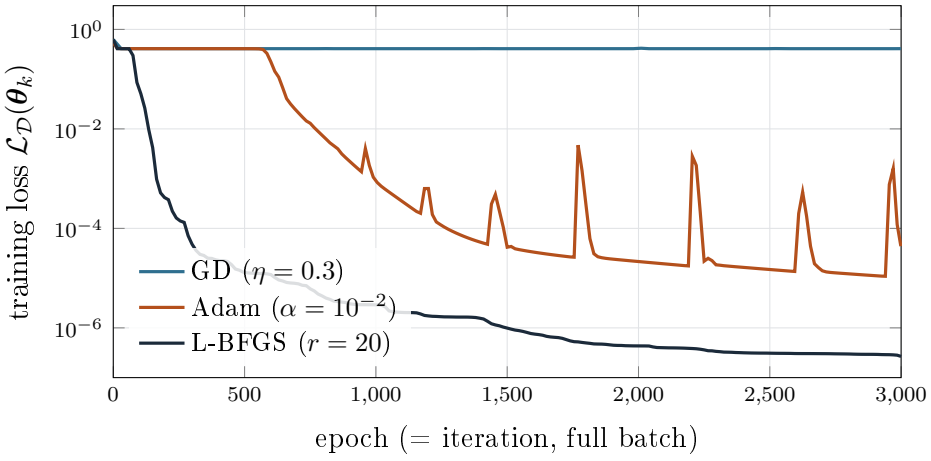


Figure 12: Training loss of GD, Adam and L-BFGS on the fit of $\sin(2\pi x)$ by an MLP with $d = 481$ parameters (full batch, same initialization).

Figure 12 shows the training loss. All three methods quickly reach a plateau ($\mathcal{L} \approx 0.41$) where the network only captures a crude trend of f. GD stays there for the whole run: its fixed, uniform step cannot leave this flat region. Adam escapes it after about 630 epochs and reaches $\mathcal{L} \approx 10^{-5}$–$10^{-4}$, with occasional spikes caused by its fixed step size (a decaying schedule, Section 3.6, would remove them). L-BFGS escapes after about 90 iterations and ends at $\mathcal{L} \approx 3 \times 10^{-7}$, about two orders of magnitude below Adam; its 3000 iterations required 3417 loss and gradient evaluations, the extra ones coming from the line search. This is the behavior that motivates the Adam $\rightarrow$ L-BFGS strategy for PINNs. The NumPy script that produces this figure is provided with these notes.

3.9 Mini-batches

Principle. All methods above use the full gradient $\nabla \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}} J\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i\big)$, which costs N forward and backward passes per iteration, prohibitive when N is large. Instead, the training set is shuffled and split into disjoint packets of B samples, the *mini-batches* $I_1, \dots, I_M$, with $M = \lceil N/B \rceil$ (Fig. 13). The parameters are updated after each mini-batch, using the loss restricted to it:

$$\mathcal{L}_{I_j}(\boldsymbol{\theta}) = \frac{1}{B} \sum_{i \in I_j} J\big(\tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i\big), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla \mathcal{L}_{I_j}(\boldsymbol{\theta}),$$

here written as a GD step; any optimizer of Section 3.5 can be used instead. One pass over all the mini-batches is one **epoch**: it now contains M parameter updates instead of one (Algorithm 2).

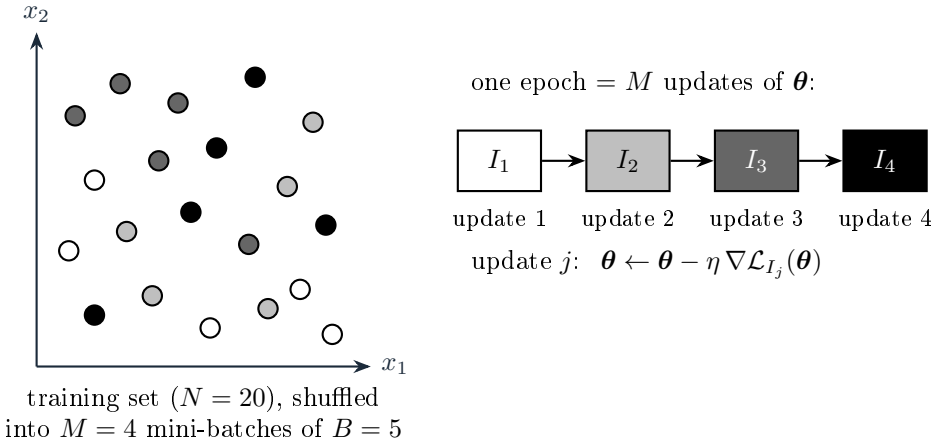


Figure 13: Mini-batches: the training set is shuffled and split into M disjoint packets (shades from white to black); the parameters are updated once per packet, so M times per epoch.

Algorithm 2: Mini-batch training

Input: training set $\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{y}_i\}_{i=1}^{N}$; initial $\boldsymbol{\theta}_0$; batch size B; number of epochs E; an optimizer (GD, Adam, ...)

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$, $M \leftarrow \lceil N/B \rceil$;

for $e = 1$ **to** E **do**

  draw a random permutation of $[\![1, N]\!]$ and cut it into M disjoint mini-batches $I_1, \dots, I_M$;

  for $j = 1$ **to** M **do**

    $\boldsymbol{g} \leftarrow \nabla \mathcal{L}_{I_j}(\boldsymbol{\theta})$; `# forward and backward pass on B samples only`

    $\boldsymbol{\theta} \leftarrow$ optimizer step$(\boldsymbol{\theta}, \boldsymbol{g})$; `# e.g. θ − η g, or Algorithm 1`

Output: $\boldsymbol{\theta}$

Properties.

- **Unbiased**: if I_j is drawn uniformly, $\mathbb{E}\big[\nabla\mathcal{L}_{I_j}(\boldsymbol{\theta})\big] = \nabla\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta})$, with a variance that decreases like $1/B$.
- **More updates, but noise**: the parameters are updated M times per epoch instead of once, at the same total cost, so the loss usually decreases much faster per epoch. In exchange, each step follows a noisy estimate of the gradient, so the loss oscillates from one iteration to the next and keeps fluctuating near the minimum (Fig. 14).
- **Names**: $B = N$ is (full-batch) GD; $B = 1$ is **Stochastic Gradient Descent (SGD)**; $1 < B < N$ is mini-batch GD, also commonly called SGD.
- **Choice of B**: typically $B = 2^k$ $(32, 64, 128, \dots)$, which suits GPU memory. Small B: cheap, noisy steps (the noise helps escape saddle points); large B: accurate gradients, better parallelism, more memory. Example: MNIST, $N = 60\,000$, $B = 128$: 469 iterations per epoch.
- **Second-order methods** need accurate gradients for their curvature estimates and line search: they are used with full batches.

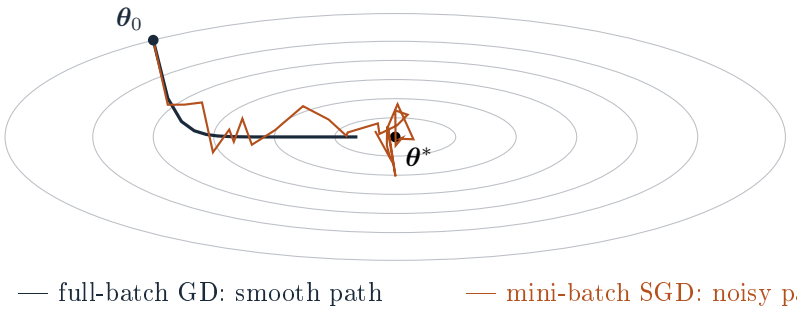


Figure 14: Same learning rate $\eta = 0.06$, 30 iterations: the mini-batch gradient is an unbiased but noisy estimate of the full gradient, so SGD wanders around the full-batch path and keeps fluctuating near $\boldsymbol{\theta}^*$.

Example. On the problem of Section 3.8, we compare Adam ($\alpha = 2 \times 10^{-3}$, 1000 epochs, same $\boldsymbol{\theta}_0$) with full batches ($B = N = 256$, one update per epoch) and with mini-batches of size $B = 32$ ($M = 8$ updates per epoch). Figure 15 shows that the mini-batch loss is noisy from one iteration to the next, but that for the same number of epochs (i.e. the same computational cost) the mini-batch version leaves the plateau much earlier (after about 380 epochs, whereas the full-batch version only starts leaving it at the end of the run) and ends at $\mathcal{L} \approx 5 \times 10^{-4}$, against 0.28 for the full-batch version.

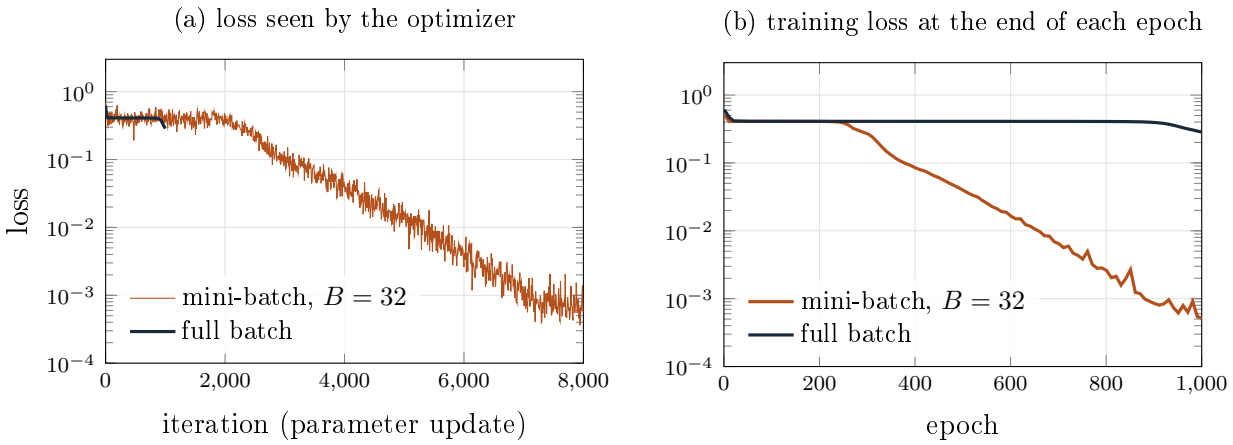


Figure 15: Adam with and without mini-batches ($\alpha = 2 \times 10^{-3}$, 1000 epochs). (a) Per iteration: the mini-batch run makes 8 updates per epoch (8000 in total, against 1000) and its loss is noisy. (b) Per epoch, i.e. at equal cost: the mini-batch run converges much faster.

3.10 Training in practice

Example: MNIST [9]. The classification of handwritten digits has $C = 10$ categories; the labels $\boldsymbol{y} = (y_1, \dots, y_C)$ are one-hot, $\hat{y}_c = \tilde{f}_{\boldsymbol{\theta}}(\boldsymbol{x})_c$, and the loss to minimize is the categorical cross-entropy $\mathcal{L}_{\text{CE}}$ of Section 3.1.

Training loop (PyTorch [10]). For each epoch, loop over the mini-batches of the training set, then evaluate on the validation set:

```
for epoch in range(n_epochs):
    # training
    model.train()                       # training mode
    for x_batch, y_batch in train_loader:
        optimizer.zero_grad()           # clear previous gradients
        loss = loss_fn(model(x_batch), y_batch)  # evaluate training loss
        loss.backward()                 # compute gradients
        optimizer.step()                # update the parameters theta

    # validation (and, at the very end, test)
    model.eval()
    with torch.no_grad():               # disable gradient evaluation
        output = model(x_val)
        val_loss = loss_fn(output, y_val)
```

→ **Use of regularization.**

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2, \qquad \lambda > 0,$$

which reduces the risk of overfitting.

References

[1] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[2] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[3] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[4] G. Hinton, N. Srivastava, K. Swersky. Neural networks for machine learning, Lecture 6e: RMSProp. Coursera lecture notes, University of Toronto, 2012.

[5] D. P. Kingma, J. Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[6] X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS*, pages 249–256, 2010.

[7] K. He, X. Zhang, S. Ren, J. Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In *Proceedings of ICCV*, pages 1026–1034, 2015.

[8] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. In *Proceedings of CVPR*, pages 770–778, 2016.

[9] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[10] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[11] D. C. Liu, J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45:503–528, 1989.

[12] J. Nocedal, S. J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[13] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

[14] A. L. Maas, A. Y. Hannun, A. Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

[15] D.-A. Clevert, T. Unterthiner, S. Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations (ICLR)*, 2016.

[16] D. Misra. Mish: a self regularized non-monotonic activation function. In *British Machine Vision Conference (BMVC)*, 2020.

[17] M. Leshno, V. Ya. Lin, A. Pinkus, S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[18] S. S. Oren, D. G. Luenberger. Self-scaling variable metric (SSVM) algorithms. Part I: criteria and sufficient conditions for scaling a class of algorithms. *Management Science*, 20(5):845–862, 1974.

[19] E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, G. E. Karniadakis. Which optimizer works best for physics-informed neural networks and Kolmogorov–Arnold networks? arXiv:2501.16371, 2025.

[20] J. Müller, M. Zeinhofer. Achieving high accuracy with PINNs via energy natural gradient descent. In *International Conference on Machine Learning (ICML)*, 2023.

[21] A. Jnini, F. Vella, M. Zeinhofer. Gauss–Newton natural gradient descent for physics-informed computational fluid dynamics. arXiv:2402.10680, 2024.

[22] A. Jnini, F. Vella. Dual natural gradient descent for scalable training of physics-informed neural networks. *Transactions on Machine Learning Research (TMLR)*, 2025.